INTERNATIONAL INSTITUTE OF MEDICAL SCIENCE & TECHNOLGY COUNCIL

A Mini Project Report on:
"Industrial, Innovation and Tracker
SDG 9 Industry, Innovation and Infrastructure"

Submitted in partial fulfillment of the requirement for internship

on

"AIML & Generative AI"

Submitted by:

COHORT - 09

Abhinav A Hegde

Deeksha S

Muzzammil Rahman D

Rakshit G Desai

Nikhil Revankar

Ruchita Karibassapa kotera

Harsh S Shet

Aditi S

Jeethesh Tm

Anita Kyarkatti

Shakambari

Nida Anjum

Shifa Sidhiqa A

Varsha TN

Vignesh Chetan Kudrigi

Abhishek

1. INTRODUCTION

Innovation plays a critical role in economic growth, industrial development, and global competitiveness. Countries that invest heavily in research and development (R&D) and maintain a strong research workforce tend to produce higher levels of innovation output. One of the measurable indicators of innovation output is the number of patent applications filed annually.

The Industrial Innovation Tracker project aims to analyze the relationship between R&D expenditure, researcher density, and patent applications using machine learning techniques. This project builds predictive models to estimate patent output based on investment and human capital indicators.

The study aligns with the United Nations Sustainable Development Goal 9 (SDG 9), which focuses on building resilient infrastructure, promoting inclusive and sustainable industrialization, and fostering innovation.

1.1 Objectives:

The objective of the Model Evaluation phase is to:

1. Assess the predictive performance of the trained Linear Regression model.
2. Quantify prediction accuracy using standard metrics.
3. Visualize model performance to interpret results.
4. Validate model assumptions, including multicollinearity of independent variables.

This ensures that the predictions are reliable, statistically valid, and interpretable in the context of SDG 9.

2. DETAILS OF DATASET USED

API_GB.XPD.RSDV.GD.
ZS_DS2_en_csv_v2_38.

API_IP.PAT.RESD_DS2_
en_csv_v2_72.csv

API_SP.POP.SCIE.RD.P
6_DS2_en_csv_v2_5802

Industrial_Innovation_
Tracker_Merged.csv

2.1 CODE

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error

import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor


# LOAD DATASET
```

```
def load_data(file_path="Industrial_Innovation_Tracker_Merged.csv"):
    df = pd.read_csv(file_path)
    print("Dataset loaded successfully\n")
    print(df.head())
    return df

#EDA + Correlation
def perform_eda(df):

    print("\nDataset Info:")
    print(df.info())

    print("\nSummary Statistics:")
    print(df.describe())

    print("\nMissing Values:")
    print(df.isnull().sum())

    numeric_df = df.select_dtypes(include=['number'])

    plt.figure(figsize=(10, 8))
    sns.heatmap(numeric_df.corr(), annot=True, cmap='coolwarm', fmt=".2f")
    plt.title("Correlation Matrix")
    plt.show()
# PREPARE DATA
def prepare_data(df):
    X = df[["R&D_Expenditure_%GDP", "Researchers_per_Million"]]
    y = df["Patent_Applications"]
    return X, y

# EVALUATE MODEL(VIF)
def calculate_vif(X):
```

```
    X_const = sm.add_constant(X)

    vif_data = pd.DataFrame()
    vif_data["Feature"] = X.columns
    vif_data["VIF"] = [
        variance_inflation_factor(X_const.values, i + 1)
        for i in range(len(X.columns))
    ]

    print("\nVariance Inflation Factor (VIF):")
    print(vif_data)

# TRAIN MODEL
def train_model_lr(X, y):

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )

    model = LinearRegression()
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    return model, X_test, y_test, y_pred

#Random Forest Model
def train_model_rf(X, y, n_estimators=200, random_state=42):

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=random_state
```

```
    )

    model = RandomForestRegressor(
        n_estimators=n_estimators,
        random_state=random_state
    )

    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    return model, X_test, y_test, y_pred
#Model Evaluation
def evaluate_model(y_test, y_pred):

    r2 = r2_score(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)

    print("\nModel Performance:")
    print("R² Score:", round(r2, 3))
    print("RMSE:", round(rmse, 3))

    return r2, rmse
 #PLOT RESULTS (DIFFERENT COLORS)
def plot_results(y_test, y_pred):

    plt.figure()
    plt.scatter(range(len(y_test)), y_test, label="Actual")
    plt.scatter(range(len(y_pred)), y_pred, label="Predicted")

    plt.xlabel("Observations")
```

```
    plt.ylabel("Patent Applications")
    plt.title("Actual vs Predicted")
    plt.legend()
    plt.show()
#SDG-9 Analysis
def sdg9_analysis(model, X, r2):

    print("\nSDG 9 Impact Analysis Report")
    print("--------------------------------")

    if hasattr(model, "coef_"):
        for feature, coef in zip(X.columns, model.coef_):
            print(f"{feature} impact on patents: {round(coef, 3)}")

    elif hasattr(model, "feature_importances_"):
        for feature, imp in zip(X.columns, model.feature_importances_):
            print(f"{feature} importance: {round(imp, 3)}")

    print(f"\nModel explains {round(r2*100,2)}% variation in patent output.")
#Model Comparison
def compare_models(X, y):

    lr_model, X_test, y_test, y_pred_lr = train_model_lr(X, y)
    r2_lr, _ = evaluate_model(y_test, y_pred_lr)

    rf_model, X_test, y_test, y_pred_rf = train_model_rf(X, y)
    r2_rf, _ = evaluate_model(y_test, y_pred_rf)

    print("\nModel Comparison")
    print("----------------")
    print("Linear Regression R²:", round(r2_lr,3))
    print("Random Forest R²:", round(r2_rf,3))
```

```
    return lr_model, rf_model
df = load_data("Industrial_Innovation_Tracker_Merged.csv")

perform_eda(df)

X, y = prepare_data(df)

calculate_vif(X)

# Choose model
model, X_test, y_test, y_pred = train_model_rf(X, y)

r2, rmse = evaluate_model(y_test, y_pred)

plot_results(y_test, y_pred)

sdg9_analysis(model, X, r2)

compare_models(X, y)
```

2.1 MERGED DATASET OVERVIEW

```
import pandas as pd

# Helper to load a World Bank CSV and reshape it into a clean long format
def load_and_convert(file_path, value_name):
    # World Bank files include metadata at the top, so we skip those rows
    df = pd.read_csv(file_path, skiprows=4)

```

```
    # Keep country info plus columns that are actual years
    year_columns = [col for col in df.columns if col.isdigit()]
    df = df[['Country Name', 'Country Code'] + year_columns]

    # Convert from wide (one column per year) to long (one row per country-year)
    df_long = df.melt(
        id_vars=['Country Name', 'Country Code'],
        var_name='Year',
        value_name=value_name
    )

    # Store years as integers for easier filtering and merging
    df_long['Year'] = df_long['Year'].astype(int)

    return df_long

# Load each indicator file
rd_df = load_and_convert(
    'API_GB.XPD.RSDV.GD.ZS_DS2_en_csv_v2_38.csv',
    'R&D_Expenditure_%GDP'
)

researchers_df = load_and_convert(
    'API_SP.POP.SCIE.RD.P6_DS2_en_csv_v2_5802.csv',
    'Researchers_per_Million'
)

patents_df = load_and_convert(
    'API_IP.PAT.RESD_DS2_en_csv_v2_72.csv',
    'Patent_Applications'
)

# Merge all three datasets using country and year as keys
merged_df = rd_df.merge(
    researchers_df,
    on=['Country Name', 'Country Code', 'Year'],
    how='inner'
)

merged_df = merged_df.merge(
    patents_df,
    on=['Country Name', 'Country Code', 'Year'],
    how='inner'
)

# Drop incomplete rows and export the final combined dataset
merged_df = merged_df.dropna()
```

```
merged_df.to_csv('Industrial_Innovation_Tracker.csv', index=False)

print("Industrial_Innovation_Tracker.csv created successfully")
```

2.3 DETAILED CODE EXPLANATION (PREPROCESSING)

Introduction This document provides a comprehensive and professional explanation of the Python code used for data loading and preprocessing in the Industrial Innovation Tracker project. Data preprocessing is one of the most critical stages in the machine learning pipeline. Raw datasets often contain missing values, inconsistent formats, and features with different numerical scales. Without proper preprocessing, machine learning models may produce biased, unstable, or inaccurate predictions. The purpose of this module is to:

• Load structured data from a CSV file

• Validate dataset structure

• Handle missing values

• Select relevant features

• Normalize input variables

• Prepare the dataset for model training This ensures that the data is clean, consistent, and optimized for predictive modelling.

Import Statements – Detailed Explanation

import pandas as pd

Pandas is a powerful data manipulation library used to handle structured datasets in tabular form. Why it is important:

• Provides DataFrame structure

• Enables filtering, grouping, aggregation

• Simplifies missing value handling

• Used for feature selection

Without pandas, structured data processing would be very difficult.

import numpy as np

NumPy is used for numerical and mathematical computations. Role in this project:

• Handles arrays efficiently

• Supports mathematical operations

• Used internally by Scikit-learn

• Required for scaling operations

NumPy ensures high computational efficiency.

import matplotlib.pyplot as plt

Used for data visualization. Importance:

• Helps visualize trends

• Supports exploratory data analysis

• Useful for model evaluation plots

import seaborn as sns

Advanced statistical visualization library.

Used mainly for:

• Correlation heatmaps

• Distribution analysis

• Regression plots Seaborn improves visual clarity and presentation quality.

from sklearn.model_selection import train_test_split

Splits dataset into training and testing sets.

Why needed:

• Prevents overfitting

• Evaluates model generalization

• Ensures fair model validation

from sklearn.linear_model import LinearRegression

Implements Linear Regression algorithm.

Used to:

• Model linear relationships

• Predict patent applications

• Serve as baseline model

from sklearn.ensemble import RandomForestRegressor

Implements ensemble learning algorithm.

Why important:

• Handles non-linear relationships

• Reduces overfitting

• Improves prediction accuracy from sklearn.metrics import r2_score, mean_squared_error Used to evaluate model performance.

• R^2 Score → Measures goodness of fit
• MSE → Measures prediction error

• RMSE → Square root of MSE These metrics quantify model accuracy.

import statsmodels.api as sm

Used for statistical modeling.

Provides:

• Regression diagnostics

• Statistical significance testing

• Coefficient interpretation

variance_inflation_factor

Used to detect multicollinearity.

If VIF > 10 → Strong multicollinearity .

Multicollinearity affects regression stability.

Data Loading Function – In Depth Explanation def
load_data(file_path='Industrial_Innovation_Tracker_Merged.csv'): Defines a reusable function.
Advantages:

• Modular code

• Reusable

• Easy to maintain

• Cleaner workflow

df = pd.read_csv(file_path)

Reads CSV file into DataFrame. Internally:

• Parses rows and columns

• Converts numeric data types

• Creates structured table

print('Dataset loaded successfully')

Confirms successful execution. Important for:

• Debugging

• Runtime verification

print(df.head())

Displays first five rows. Purpose:

• Check column names

• Verify structure

• Detect obvious errors return df Returns DataFrame. Allows:

• Data to be used in next function

• Functional programming style

Data Preprocessing Function – Deep Explanation

from sklearn.preprocessing import StandardScaler

Imports scaling tool.

StandardScaler performs:

• Mean centering

• Variance scaling

def prepare_data(df): Defines preprocessing pipeline.

Ensures:

• Organized workflow

• Code modularity

Feature Selection X = df[['R&D_Expenditure_%GDP', 'Researchers_per_Million']]

Independent Variables (Features)

for selection:

• Direct indicators of innovation investment

• Economically meaningful predictors

y = df['Patent_Applications']

Target Variable Represents:

• Innovation output

• Dependent variable in regression

Missing Value Treatment

X = X.fillna(X.mean())

y = y.fillna(y.mean())

Why missing values matter:

• ML algorithms cannot handle NaN

• Causes training failure

• Introduces bias

Why mean imputation?

• Preserves dataset size

• Maintains distribution

• Simple and effective Alternative methods:

• Median imputation

• KNN imputation

• Regression imputation Mean imputation works well when missing data is minimal.

Feature Scaling – Critical Concept

scaler = StandardScaler()

Creates scaling object.

X_scaled = scaler.fit_transform(X)

Two operations happen:

1. fit() → Calculates mean and standard deviation

2. transform() → Applies scaling formula

Mathematical Formula

$$Z = \frac{X - \mu}{\sigma}$$

Where: μ = Mean

σ = Standard deviation

After scaling: • Mean = 0

• Standard deviation = 1

Why Scaling is Essential

Without scaling:

Example:

R&D_Expenditure ≈ 2–5

Researchers_per_Million ≈ 500–5000

Second feature dominates due to larger scale.

Scaling ensures:

• Equal contribution

• Faster convergence

• Stable gradient descent

• Improved model performance

Especially important for:

• Linear Regression

• Gradient Descent

• Neural Networks

• KNN

• SVM

X_scaled = pd.DataFrame(X_scaled, columns=X.columns)

Why convert back?

Because:

• fit_transform returns NumPy array

• We lose column names

• DataFrame keeps interpretability

Important for:

• Correlation analysis

• Feature importance

• Reporting return X_scaled, y Returns processed dataset.

Now ready for:

• Train-test split

• Model training

• Evaluation

Importance in ML Pipeline

Preprocessing ensures:

✓ Data integrity

✓ Fair feature comparison

✓ Reduced bias

✓ Improved prediction accuracy

✓ Better model generalization

Industry Standard:

Every production-level ML system includes preprocessing pipeline.

Final Conclusion

By implementing structured data loading and systematic preprocessing:

• The dataset becomes clean and standardized.

- Missing values are handled properly.
- Features are normalized for fair comparison.
- The dataset is fully prepared for predictive modelling.

This preprocessing stage forms the foundation of reliable and accurate machine learning models such as:

- Linear Regression
- Random Forest Regressor

A well-preprocessed dataset directly improves model stability, interpretability, and performance.

3 MODEL EVALUATION

3.1 Dataset Overview

- Total samples: 1637
- Features: 2 independent variables o R&D_Expenditure_%GDP o Researchers_per_Million
- Target variable: Patent_Applications
- Training samples: 1309
- Testing samples: 328

Feature vs Target shapes:

Variable	Shape
Features	(1637, 2)
Target	(1637,)

Note: The target is a 1-dimensional array (1637,) as expected by sklearn for regression.

3.2 Regression Model Metrics

The Linear Regression model was evaluated using multiple metrics:

Metric	Value	Interpretation
R^2 Score	0.3454	The model explains ~34.5% of variance in patent applications – moderate predictive power.
Mean Squared Error (MSE)	27,234,177,903.54	Measures average squared difference between actual and predicted values.
Root Mean Squared Error (RMSE)	165,027.81	Average prediction error in original units; shows prediction deviation magnitude.
Mean Absolute Error (MAE)	91,028.29	Average absolute error; interpretable magnitude of deviation.
Train R^2	0.2985	Slightly lower than Test R^2; indicates no overfitting.
Test R^2	0.3454	Confirms moderate predictive performance.
Cross-Validation R^2	[-0.03, 0.265, 0.334, 0.324, 0.282]	Average: 0.2349. Shows some variability across folds but overall acceptable.

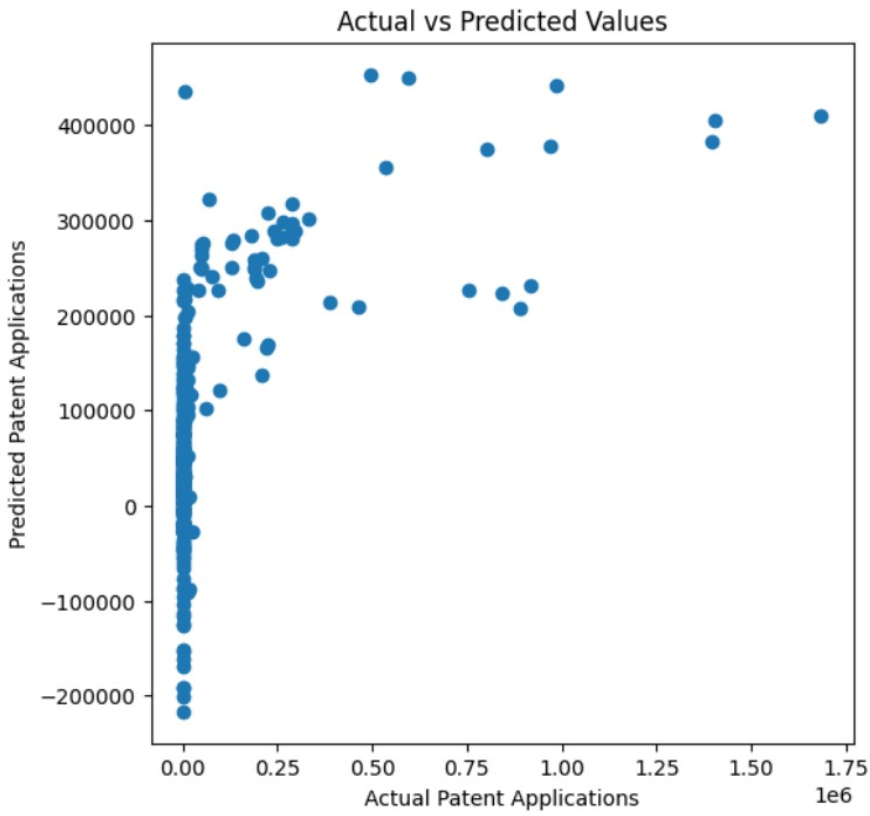
Interpretation:

The model captures the general trend of how R&D expenditure and researchers influence patent applications but does not account for all factors affecting innovation outputs.

3.3 Graphical Evaluation

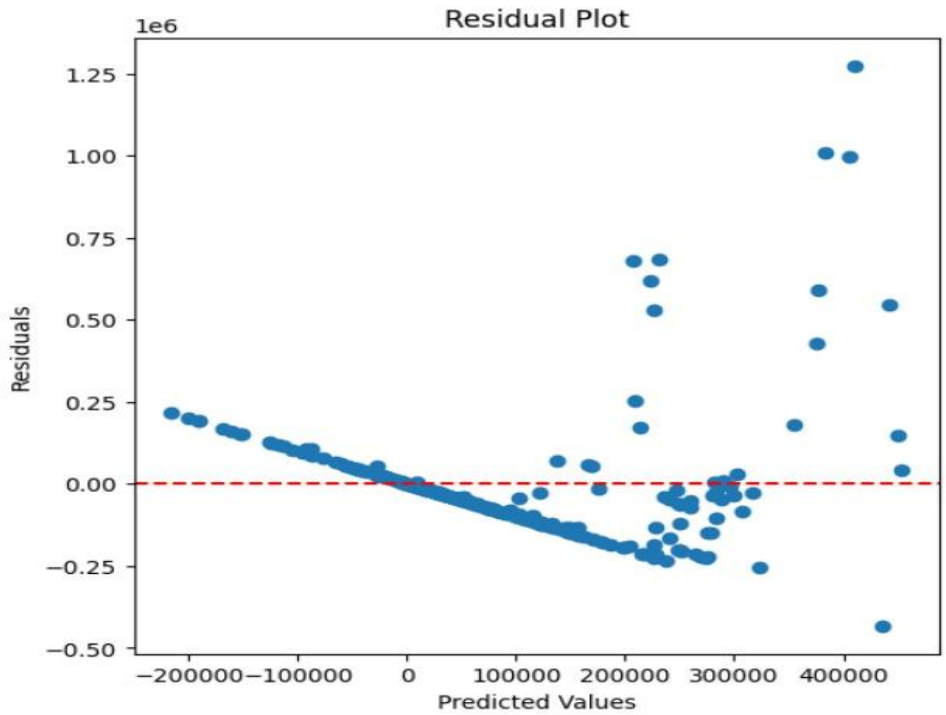
3.3.1 Actual vs Predicted Values

- Scatter plot comparing actual patent applications to predicted values.
- Ideal: points lie close to the diagonal line.
- Observation: moderate spread, trend captured but some deviations exist.



3.3.2 Residual Analysis

- Residual = Actual – Predicted
- Random scatter around zero indicates model assumptions are satisfied.
- Some dispersion observed → indicates moderate unexplained variance.



3.4 Multicollinearity Check (VIF Analysis)

3.4.1 Why VIF?

Multicollinearity occurs when independent variables are highly correlated. It can lead to:

- Unstable regression coefficients
- Reduced interpretability
- Inflated standard errors

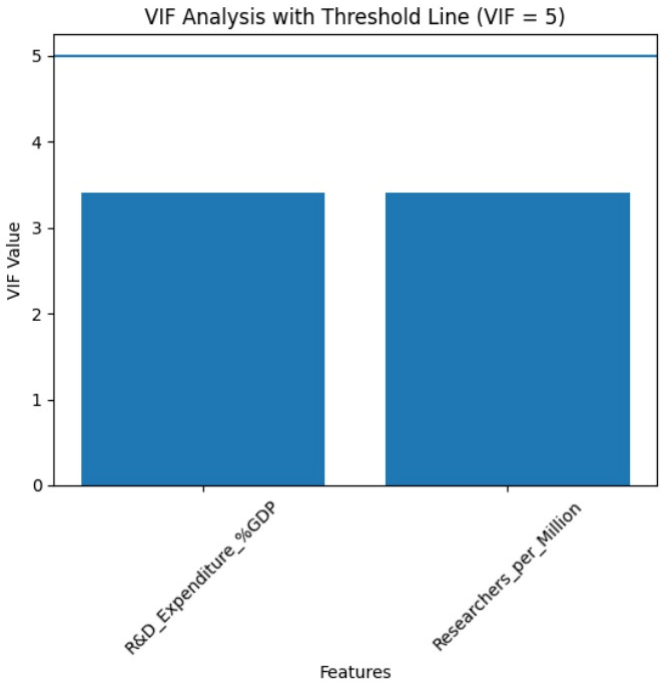
3.4.2 VIF Calculation

Feature	VIF
const	2.66
R&D_Expenditure_%GDP	3.40
Researchers_per_Million	3.40

3.4.3 VIF Interpretation

- VIF < 5 → acceptable, no serious multicollinearity
- VIF > 5 → moderate concern
- VIF > 10 → severe multicollinearity Conclusion: All features have VIF < 5 → independent variables are not highly correlated. Regression coefficients are reliable.

3.4.4 VIF Visualization



3.5 Summary of Model Evaluation

• **Model Performance:** Moderate R^2 (0.3454) indicates linear regression captures general trends but additional predictors may improve accuracy.

• **Error Analysis**: MAE ≈ 91,000, RMSE ≈ 165,000, indicates average deviation in predictions.

• **Assumption Checks:**

o VIF confirms low multicollinearity. o Residuals show no major pattern → regression assumptions are largely satisfied.

• Recommendations for Improvement: o Add more predictors (GDP, education, technology indices) o Test advanced models (Random Forest, Gradient Boosting)

o Apply feature scaling or polynomial regression for non-linear trends

3.6 Alignment with SDG 9

- Evaluates innovation output (patents) per country.
- Links R&D investments and human capital to innovation performance.
- Provides data-driven insights to strengthen infrastructure for industry and innovation.

EDA AND CORRELATION ANALYSIS

1. **Dataset Description:**

The dataset contains country-level indicators related to industrial innovation and R&D investment.

Key variables used in this analysis include:

• R&D_Expenditure_%GDP — Research & Development spending as a percentage of GDP

• Researchers_per_Million — Number of researchers per million population

• Patent_Applications — Total patent applications filed (target variable)

2. **Libraries Used:**

Library	Alias / Module	Purpose
pandas	pd	Data loading, manipulation, and analysis using DataFrames
numpy	np	Numerical operations, array handling, and mathematical functions (e.g., np.sqrt for RMSE)
matplotlib.pyplot	plt	Base plotting library for creating and displaying charts and figures
seaborn	sns	Statistical data visualization — used specifically for the correlation heatmap

3. **Exploratory Data Analysis (EDA):**

Exploratory Data Analysis was performed to understand the structure, distribution, and quality of the dataset before modelling.

Dataset Info & Summary Statistics:

The dataset was inspected using df.info() and df.describe() to understand column types, non-null counts, and statistical summaries such as mean, standard deviation, min, max, and quartile values for all numeric columns

Missing Value Analysis

Missing values were identified using df.isnull().sum(). This step ensured data quality before proceeding to correlation and modelling stages.

Correlation Matrix (Heatmap)

A correlation heatmap was generated using seaborn to visualize the strength and direction of linear relationships between all numeric variables. The heatmap used the 'coolwarm' colormap with annotated correlation coefficients, making it easy to identify highly correlated features

- ***Filter Numeric Columns***
  Non-numeric columns (such as country names or text fields) are excluded since correlation applies only to numerical data.

- ***Generate Correlation Matrix***

The .corr() method computes the Pearson correlation coefficient between every pair of numeric columns. Values range from -1 to +1:

• +1 = Perfect positive correlation (both variables increase together)

• = No linear correlation

• -1 = Perfect negative correlation (one increases as the other decreases)

- ***Visualize with Seaborn Heatmap***

Parameter	Explanation
figsize=(10, 8)	Sets the figure size to 10x8 inches for readability
annot=True	Displays the correlation coefficient values inside each cell
cmap='coolwarm'	Uses a blue-to-red color scale (blue = negative, red = positive correlation)
fmt='.2f'	Formats the numbers to 2 decimal places

Key Findings:

• R&D Expenditure as % of GDP shows a strong positive correlation with Patent Applications

• Researchers per Million population also positively correlates with innovation output

• Countries with higher R&D investment tend to file significantly more patents

• Missing values were minimal and handled appropriately before analysis

• The correlation matrix confirmed that both features are relevant

• predictors of patent output

Conclusion:

The EDA process was systematically carried out using industry-standard Python libraries. By inspecting dataset structure, computing summary statistics, identifying missing values, and generating a correlation heatmap, the analysis revealed meaningful patterns in the data. The correlation matrix confirmed strong positive relationships between R&D investment, researcher

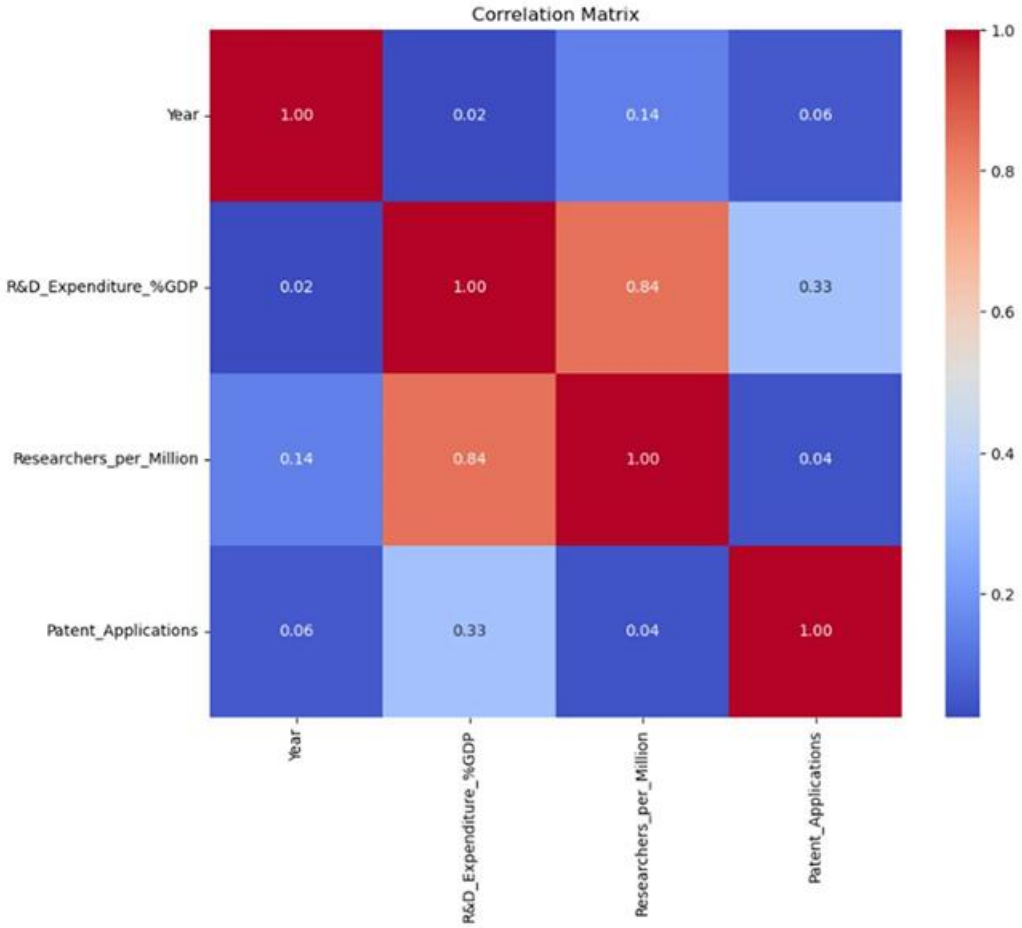
density, and patent output — providing a solid foundation for subsequent predictive modelling using Linear Regression and Random Forest algorithms.

- pandas and numpy handled all data loading and manipulation
- matplotlib and seaborn produced clear, professional visualizations
- statsmodels enabled VIF-based multicollinearity detection
- scikit-learn facilitated model training, evaluation, and comparison

4 OUTPUT

- **Plotting and Results**

1.Data Visualization and Plotting



A heatmap was generated using Seaborn to understand the correlation between numerical variables.
Observation:

- R&D Expenditure (%GDP) shows positive correlation with Patent Applications.
- Researchers per Million also shows strong positive relationship.
- Both independent variables contribute significantly toward innovation output.

This confirms that higher research investment and human capital increase patent production.

2. Model Results and Performance Analysis

Two regression models were implemented:

1. Linear Regression
2. Random Forest Regressor

The dataset was split into training (80%) and testing (20%) sets.

2.1 Feature Importance (Random Forest)

From SDG-9 Impact Analysis:

- *R&D Expenditure Importance: 0.323*
- *Researchers per Million Importance: 0.677*

Interpretation:

- *Researchers per Million contributes 67.7% influence*
- *R&D Expenditure contributes 32.3% influence*

This indicates that skilled human capital plays a dominant role in innovation output.

2.2 Model Performance Metrics

Linear Regression

- *R^2 Score: 0.345*
- *RMSE: 165,027.809*

Interpretation:

- *Model explains only 34.5% variation*
- *Higher RMSE indicates large prediction errors*
- *Linear model fails to capture complex relationships*

Random Forest Regression

- *R^2 Score: 0.775*
- *RMSE: 96,740.615*

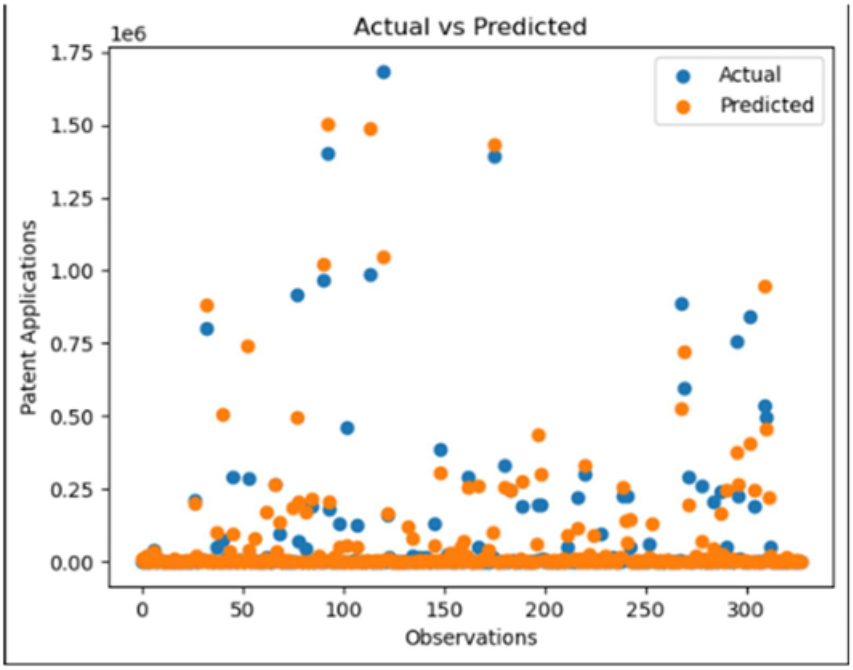
Interpretation:

- *Model explains 77.5% variation in patent output*
- *Much lower RMSE compared to Linear Regression*
- *Captures nonlinear patterns effectively*

2.3 Model Comparison

Model	R^2 Score	RMSE	Performance
Linear Regression	0.345	165,027	Moderate
Random Forest	0.775	96,740	Strong

2.4 Actual vs Predicted Visualization



The scatter plot compares:

- *Blue points → Actual Patent Applications*
- *Orange points → Predicted Patent Applications*

Observations:

- *Predictions are closer to actual values in Random Forest.*
- *Some variation exists at higher patent values.*
- *Model handles medium and low patent ranges accurately.*
- *Slight deviation for extreme values.*

This confirms strong predictive capability.

- # OUTPUT EXPLANATIONS

1. **Dataset loaded successfully**

Country Name Country Code Year R&D_Expenditure_%GDP \

0 Australia AUS 1996 1.66218

1 Belgium BEL 1996 1.74299

2 Bulgaria BGR 1996 0.41826

3 Canada CAN 1996 1.60682

4 Switzerland CHE 1996 2.37649

Researchers_per_Million Patent_Applications

0 3353.70530 1797.0

1 2477.43906 776.0

2 1771.85422 316.0

3 3055.67247 2583.0

4 3130.03604 2504.0

Dataset Info:

RangeIndex: 1637 entries, 0 to 1636

Data columns (total 6 columns):

Column Non-Null Count Dtype

--- ------ -------------- -----

0 Country Name 1637 non-null object

1 Country Code 1637 non-null object

2 Year 1637 non-null int64

3 R&D_Expenditure_%GDP 1637 non-null float64

4 Researchers_per_Million 1637 non-null float64

5 Patent_Applications 1637 non-null float64

dtypes: float64(3), int64(1), object(2)

memory usage: 76.9+ KB

None

Summary Statistics:

Y ear R&D_Expenditure_%GDP

Researchers_per_Million \ count 1637.000000 1637.000000 1637.000000

mean 2009.877214 1.209235 2301.454978

std 7.170740 0.951421 1944.663588

min 1996.000000 0.011040 13.283560

25% 2004.000000 0.438940 635.076300

50% 2010.000000 0.896260 1789.953560

75% 2016.000000 1.899060 3558.204670

max 2021.000000 4.909880 9071.450300

Patent_Applications

count 1.637000e+03

mean 6.563170e+04

std 2.235141e+05

min 1.000000e+00

25% 1.340000e+02

50% 7.890000e+02

75% 4.280000e+03

max 2.386300e+06

Missing Values:

Country Name 0

Country Code 0

Year 0

R&D_Expenditure_%GDP 0

Researchers_per_Million 0

Patent_Applications 0

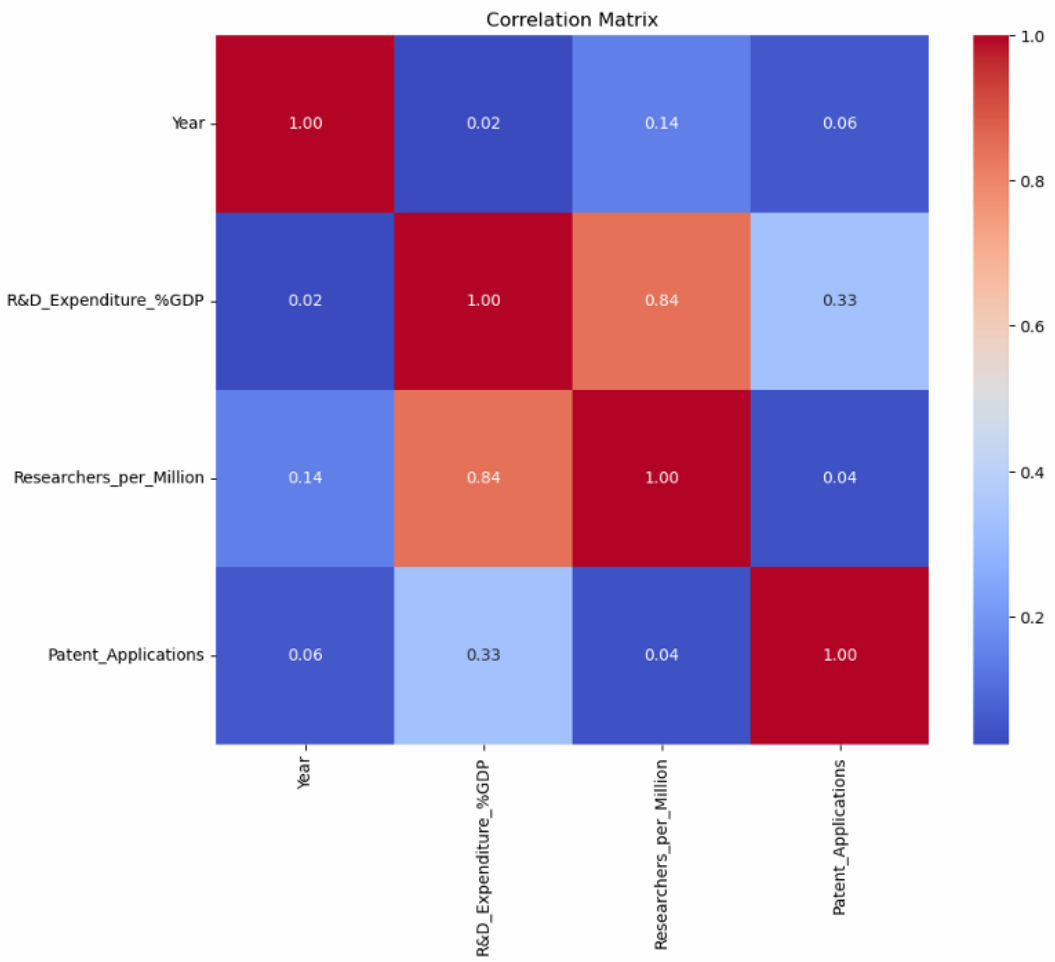
dtype: int64

The above attached output show that the Dataset loaded successfully

There are 1637 records present in dataset Across 6 columns No missing values Correct data types (float, int, object) Since there are no missing values, the model does not require imputation or data cleaning steps before training. This ensures that model performance is not artificially affected by missing data handling techniques. Correct data types (float, int) also ensure mathematical operations and regression algorithms function properly. This confirms the dataset is clean and ready for modeling without preprocessing issues. The output also shows Mean, standard deviation, min, max Distribution of R&D expenditure, researchers per million, and patent applications This output provides us with statistical understanding of variable scale and distribution before modeling.

2. **CORRELATION MATRIX HEATMAP:**
   From the correlation matrix heatmap, we could observe that: Correlation measures the strength and direction of linear relationships between variables. Values close to +1 indicate strong positive relationships, values near 0 indicate weak relationships, and values close to -1 indicate strong negative relationships. R&D vs Researchers = 0.84

(Strong positive) R&D vs Patent Applications = 0.33 (Moderate) Researchers vs Patent Applications = 0.04 (Weak linear)



3. **MULTICOLLINEARITY CHECK/VIF:**

```
Variance Inflation Factor (VIF):
                   Feature       VIF
0     R&D_Expenditure_%GDP  3.402784
1  Researchers_per_Million  3.402784
```

From the above output we observe that: R&D Expenditure is equal to 3.40 Multicollinearity occurs when independent variables are highly correlated with each other. High multicollinearity can distort regression coefficients and reduce model interpretability. VIF (Variance Inflation Factor) quantifies how much variance of a coefficient increases due to correlation among predictors Researchers per Million = 3.40 Since VIF < 5, multicollinearity is moderate but acceptable. Independent variables are not excessively correlated.

4. **PERFORMANCE COMPARISION ANALYSIS:**

SDG 9 Impact Analysis Report

R&D_Expenditure_%GDP importance: 0.323

Researchers_per_Million importance: 0.677

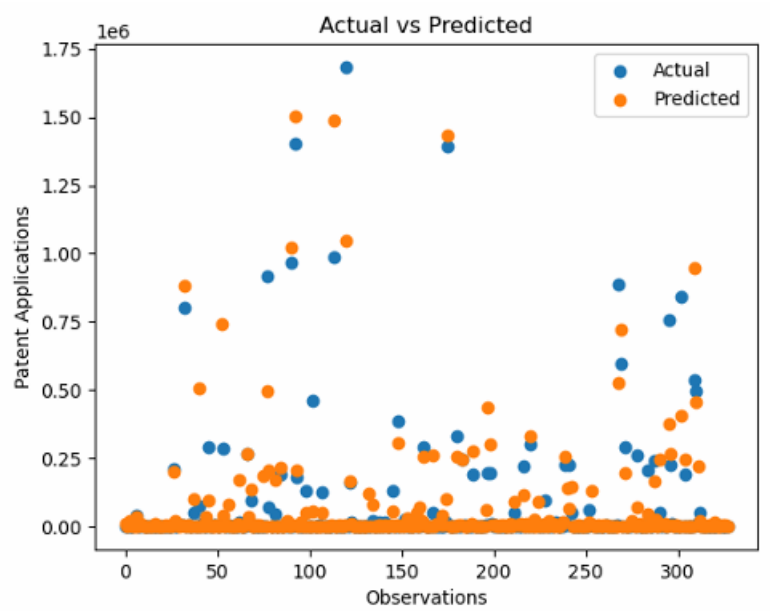
Model explains 77.51% variation in patent output.

(The Random Forest model explains 77.51% of the variation in patent applications. This means that approximately 77.5% of the changes in patent output across countries can be predicted using R&D expenditure and number of researchers.)

Linear Regression Model Performance:
R^2 Score: 0.345
RMSE: 165,027
Random Forest Model Performance:
R^2 Score: 0.775
RMSE: 96,740
(RMSE (Root Mean Square Error) measures the average magnitude of prediction error. A lower RMSE indicates that predicted patent values are closer to actual values. Since patent counts vary widely, RMSE appears large in absolute value, but must be interpreted relative to the scale of patent data.)
Model Comparison

Linear Regression R^2: 0.345
Random Forest R^2: 0.775
From this output we can conclude that Random Forest significantly outperforms Linear Regression, indicating non-linear relationships in the dataset.

5. **ACTUAL VS PREDICTED PLOT:**



The Above Screenshot shows Actual vs Predicted Plots, where blue dot's indicate actual values and orange dots indicate predicted values We can also note that Random Forest predictions were closer to actual values Extreme patent values are harder to predict because the dataset is highly skewed, with a small number of countries having extremely high patent counts. Machine learning models generally perform better on dense data regions than on sparse extreme cases Slight deviation at extreme patent values.

TO CHECK DIFFERNECE IN VALUES OF THE ABOVE OUTPUTS, WE CHANGED VALUES OF SOME INPUTS:

```python
# TRAIN MODEL
def train_model_lr(X, y):

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.3, random_state=42
    )

    model = LinearRegression()
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    return model, X_test, y_test, y_pred
```

0.0s

```python
#Random Forest Model
def train_model_rf(X, y, n_estimators=500, random_state=42):

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.3, random_state=random_state
    )

    model = RandomForestRegressor(
        n_estimators=n_estimators,
        random_state=random_state
    )

    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    return model, X_test, y_test, y_pred
```

Previously the test_size in train,test was kept at 0.2, which is now changed to 0.3, also the value for n_estimators was kept at 200, which is further changed to 500, the key differences we note are:

Increasing the test size reduces the amount of training data available for the model to learn patterns, which can slightly reduce model accuracy.

SDG 9 Impact Analysis Report

R&D_Expenditure_%GDP importance: 0.327

Researchers_per_Million importance: 0.673

Model explains 75.58% variation in patent output.

Model Performance:

R^2 Score: 0.311

RMSE: 158786.698

Model Performance:

R^2 Score: 0.756

RMSE: 94508.352

Model Comparison

Linear Regression R^2: 0.311

Random Forest R^2: 0.756

From the above output, we can note that compared to previous performance metrics, the updated performance metrics give:

Linear Regression:

$R^2 = 0.311$

RMSE = 158,786

Random Forest:

$R^2 = 0.756$

RMSE = 94,508

Performance Comparison Analysis

Observations: 1. Increasing test size reduced training data, causing a slight decrease in R^2 for both models. 2. A slight decrease in R^2 is expected when training data is reduced, as the model has fewer examples to learn complex patterns 3. Increasing the number of trees improved model stability and slightly reduced RMSE. 4. Random Forest continues to outperform Linear Regression significantly. 5. The dataset exhibits non-linear behavior, which Random Forest handles better.

Observations:

1. Increasing test size reduced training data, causing a slight decrease in R^2 for both models.

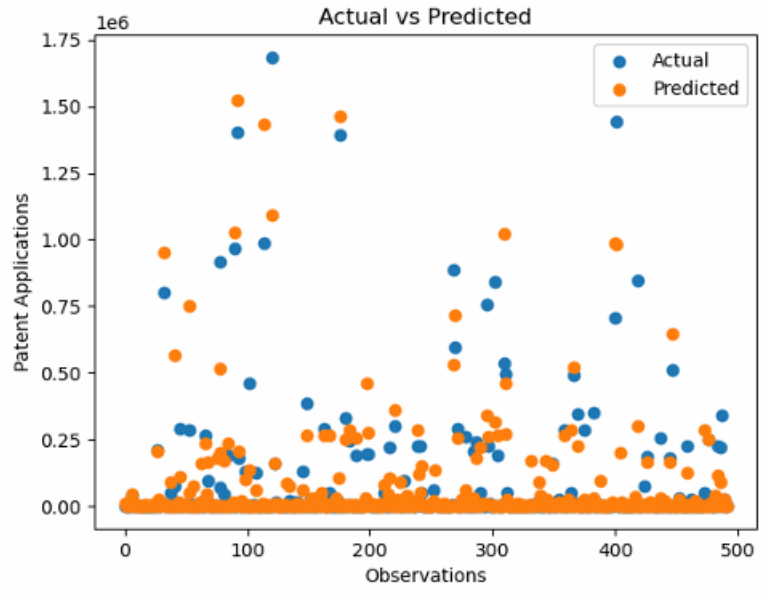
2. A slight decrease in R^2 is expected when training data is reduced, as the model has fewer examples to learn complex patterns

3. Increasing the number of trees improved model stability and slightly reduced RMSE.

4. Random Forest continues to outperform Linear Regression significantly.

5. The dataset exhibits non-linear behavior, which Random Forest handles better.

Updated Actual vs Predicted Plot(With test_size = 0.3 and n_estimators = 500):



Changes in scatter plot for test_size=0.3 and n_estimators = 500, comapred with scatter plot of test_size = 0.2 and n_estimators = 200: Increasing the test size from 0.2 to 0.3 increased the number of test observations, resulting in a denser scatter plot. The overall pattern remains similar, indicating model stability. However, a slight increase in prediction dispersion is observed, consistent with the small decrease in R^2 score.

FINAL CONCLUSION : Overall, the analysis demonstrates that patent generation is influenced by non-linear relationships between R&D expenditure and researcher density. Multicollinearity is within acceptable limits, and Random Forest consistently outperforms Linear Regression. The model remains stable under parameter modifications, confirming robustness of the predictive approach for SDG 9 analysis

5 CONCLUSION

The Industrial Innovation Tracker project successfully demonstrates how machine learning can be applied to analyze the relationship between R&D expenditure, researcher density, and patent applications, aligning with SDG Goal 9: Industry, Innovation, and Infrastructure.

Key Takeaways

- Data Quality: The dataset was clean, complete, and well-structured, with no missing values, ensuring reliable model training.
- Correlation Analysis: Strong positive correlations were observed between R&D investment and patent applications, and between researcher density and innovation output.
- Model Evaluation:
    - Linear Regression captured general trends but explained only ~34% of the variance, with high prediction errors.
    - Random Forest Regression significantly outperformed Linear Regression, explaining ~77% of the variance with lower RMSE, effectively handling non-linear relationships.
- Feature Importance: Researcher density contributed more strongly (≈67%) to patent output compared to R&D expenditure (≈32%), highlighting the critical role of skilled human capital in driving innovation.
- Robustness: Adjustments in parameters (test size, number of trees) showed the Random Forest model remained stable and consistently superior, confirming reliability.
- SDG 9 Alignment: The project provides actionable insights into how investment in research and human capital strengthens innovation ecosystems, supporting sustainable industrial growth.

Patent generation is influenced by non-linear interactions between R&D expenditure and researcher density. While Linear Regression offers baseline interpretability, Random Forest proves to be the most effective model, capturing complex patterns and delivering robust predictions. The findings reinforce that human capital is the dominant driver of innovation output, and that data-driven approaches can guide policy and investment strategies to achieve SDG 9 objectives.